

Find Redundant Directed Connection

Authored by: Arpit Raj, LNMIIT Jaipur | Date: August 3, 2026

Theoretical Notes

A valid rooted tree is:-

- (i) every node has exactly one parent i.e. indegree = 1 (except root node)
- (ii) No cycle

One extra edge has been added, what could go wrong ->

- (i) only cycle
- (ii) Indegree 2
- (iii) both

Logic & Cases

Case 1: Indegree 2 is missing

If no node has indegree 2, then simply return the first edge that creates a cycle.

Case 2: Node has Indegree 2

Example: $1 \rightarrow 3 \leftarrow 2$

Now we only have 2 candidates:

cond 1: $1 \rightarrow 3$

cond 2: $2 \rightarrow 3$

} one of them is wrong

Ignore cond 2, DSU still finds cycle which means cond 1 was wrong. return cond 1.

Similar with ignore cond 1.

C++ Implementation (with Explanation Notes)

```
class dsu {
public:
    vector<int> parent;
    vector<int> size;

    dsu(int n){
        parent.resize(n);
        size.resize(n, 1);

        for(int i = 0; i < n; i++){
            parent[i] = i;
        }
    }

    int findparent(int node){
        if(parent[node] == node){
            return node;
        }
        parent[node] = findparent(parent[node]);
        return parent[node];
    }

    void unionit(int u, int v){
        int pu = findparent(u);
```

```

    int pv = findparent(v);

    if(pu == pv) return;

    if(size[pu] < size[pv]) swap(pu, pv);

    parent[pv] = pu;
    size[pu] += size[pv];
}
};

class Solution {
public:
    vector<int> findRedundantDirectedConnection(vector<vector<int>>& edges) {

        // dsu approach lagayenge
        // sabse pehle to rooted tree hone ke liye
        // indegree==1 sbke liye except node
        // no cycle
        // exactly ek edge add hua he iss property ko destroy krne ke liye
        // to sbse pehle sabki indegree nikaal lenge
        // agar kisi ki indegree 2 nhi hui matlab cycle bana hua he
        // pehla edge jo cycle bana rha wo return

        // agar indegree 2 ho rhi to do candidates honge
        // candidate 2 ignore krne ke baad bhi cycle ban rhi to candidate 1 answer
        // else candidate 2 answer

        int n = edges.size();
        vector<int> indeg(n + 1, 0);
        for(auto &it : edges){
            indeg[it[1]]++;
        }

        int sus = -1;
        for(int i = 1; i <= n; i++){
            if(indeg[i] == 2){
                sus = i;
            }
        }

        if(sus == -1){
            // matlab kisiki indeg 2 nhi thi
            // koi cycle exist kr rha higa bas

```

```

dsu ds(n + 1);
for(auto &it : edges){
    int u = it[0];
    int v = it[1];

    if(ds.findparent(u) == ds.findparent(v)){
        return it;
    }

    ds.unionit(u, v);
}
}

```

*// otherwise kisi ek node ke do baap he , wo dono edge hame chahiye
// unme se kisi ek ko remove krne ke baad problem solve hogi*

```

vector<int> chutiya1;
vector<int> chutiya2;
for(auto &it : edges){
    int u = it[0];
    int v = it[1];

    if(v == sus){
        if(chutiya1.empty()){
            chutiya1 = it;
        }
        else{
            chutiya2 = it;
        }
    }
}
}

```

*// ek baar chutiya2 ko ignore krke dekh lete
// uske bawajood cycle ban rhi to chutiya1 answer
// else chutiya2 answer*

```

dsu ds(n + 1);
for(auto &it : edges){
    if(it == chutiya2) continue;
    int u = it[0];
    int v = it[1];

    if(ds.findparent(u) == ds.findparent(v)){
        return chutiya1;
    }
}

```

```
        ds.unionit(u, v);  
    }  
  
    return chutiya2;  
}  
};
```